

Souvenirs

Amaru achète des souvenirs dans un magasin. Il y a N **types** de souvenirs. Il y a une infinité de souvenirs de chaque type disponible dans le magasin.

Chaque type de souvenir est vendu à un prix fixe. C'est-à-dire, un souvenir de type i ($0 \leq i < N$) coûte $P[i]$ pièces, où $P[i]$ est un nombre entier strictement positif.

Amaru sait que les types de souvenirs sont classés par ordre de prix décroissants et que les prix des types de souvenirs sont distincts. En particulier, $P[0] > P[1] > \dots > P[N-1] > 0$. De plus, il connaît la valeur de $P[0]$. Malheureusement, Amaru ne connaît aucune autre information concernant le prix des souvenirs.

Pour acheter des souvenirs, Amaru effectuera un certain nombre de transactions avec le vendeur.

Chaque transaction se fait de la manière suivante :

1. Amaru donne un certain nombre de pièces (strictement positif) au vendeur.
2. Le vendeur empile ces pièces sur une table dans la pièce du fond où Amaru ne peut pas les voir.
3. Le vendeur considère chaque type de souvenir, l'un après l'autre, $0, 1, \dots, N-1$ dans cet ordre. Chaque type est pris en compte **exactement une fois** par transaction.
 - Lorsqu'il considère le type de souvenir i , si le nombre actuel de pièces dans la pile est au moins égal à $P[i]$, alors
 - le vendeur retire $P[i]$ pièces de la pile, et
 - le vendeur place un souvenir du type i sur la table.
4. Le vendeur donne à Amaru toutes les pièces restantes dans la pile et tous les souvenirs posés sur la table.

Notez qu'il n'y a ni pièce, ni souvenir sur la table avant le début d'une transaction.

Votre tâche consiste à demander à Amaru d'effectuer un certain nombre de transactions, de sorte que :

- à chaque transaction, il achète **au moins un** souvenir.
- au total, il achète **exactement** i souvenirs du type i , pour chaque i tel que $0 \leq i < N$. Notez que cela signifie qu'Amaru ne doit acheter aucun souvenir du type 0.

Amaru n'a pas besoin de minimiser le nombre de transactions et dispose d'une réserve illimitée de pièces.

Détails d'implémentation

Vous devez implémenter la fonction suivante.

```
void buy_souvenirs(int N, long long P0)
```

- N : le nombre de types de souvenirs
- $P0$: la valeur de $P[0]$
- Cette fonction est appelée exactement une fois pour chaque test.

La fonction ci-dessus peut appeler la fonction suivante pour demander à Amaru d'effectuer une transaction :

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : le nombre de pièces qu'Amaru donne au vendeur.
- La fonction renvoie une paire. Le premier élément de la paire est un tableau L , contenant les types de souvenirs qui ont été achetés (dans l'ordre croissant). Le second élément de la paire est un entier R , le nombre de pièces rendues à Amaru à la fin de la transaction
- Il est nécessaire que $P[0] > M \geq P[N - 1]$. La condition que $P[0] > M$ garantit qu'Amaru n'achètera pas de souvenirs de type 0, et $M \geq P[N - 1]$ garantit qu'Amaru achètera au moins un souvenir. Si ces conditions ne sont pas satisfaites, votre solution recevra le verdict `Output isn't correct: Invalid argument`. Notez que contrairement à $P[0]$, la valeur de $P[N - 1]$ n'est pas fournie dans l'entrée.
- Pour chaque test, la fonction `transaction` peut être appelée au plus 5000 fois.

Le comportement du grader **n'est pas adaptif**. Cela signifie que la suite de prix P est fixée avant l'appel de `buy_souvenirs`

Contraintes

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ pour tout i tel que $0 \leq i < N$.
- $P[i] > P[i + 1]$ pour tout i tel que $0 \leq i < N - 1$.

Sous-tâches

Sous-tâche	Score	Contraintes supplémentaires
1	4	$N = 2$
2	3	$P[i] = N - i$ pour tout i tel que $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ pour tout i tel que $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ pour tout i tel que $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ pour tout i tel que $0 \leq i < N - 1$.
6	33	Aucune contrainte supplémentaire

Exemple

Considérez l'appel suivant.

```
buy_souvenirs(3, 4)
```

Il y a $N = 3$ types de souvenirs et $P[0] = 4$. Observez qu'il n'y a que trois suites P de prix possibles : $[4, 3, 2]$, $[4, 3, 1]$, et $[4, 2, 1]$.

Supposons que `buy_souvenirs` appelle `transaction(2)` et que l'appel renvoie $([2], 1)$, signifiant qu'Amaru a acheté un souvenir du type 2 et que le vendeur lui a rendu 1 pièce. Observez que cela nous permet de déduire que $P = [4, 3, 1]$, puisque :

- Pour $P = [4, 3, 2]$, `transaction(2)` aurait renvoyé $([2], 0)$.
- Pour $P = [4, 2, 1]$, `transaction(2)` aurait renvoyé $([1], 0)$.

Ensuite, `buy_souvenirs` peut appeler `transaction(3)`, qui renvoie $([1], 0)$, signifiant qu'Amaru a acheté un souvenir de type 1 et que le vendeur lui a rendu 0 pièces. À présent, au total, Amaru a acheté un souvenir du type 1 et un souvenir du type 2.

Finalement, `buy_souvenirs` peut appeler `transaction(1)`, qui renvoie $([2], 0)$, signifiant qu'Amaru a acheté un souvenir du type 2. Notez que nous aurions également pu appeler `transaction(2)` au lieu de `transaction(1)`. À présent, au total, Amaru a acheté un souvenir du type 1 et deux souvenirs du type 2, comme demandé.

Évaluateur d'exemple (grader)

Format d'entrée :

N

P[0] P[1] ... P[N-1]

Format de sortie :

Q[0] Q[1] ... Q[N-1]

Ici, pour chaque i tel que $0 \leq i < N$, $Q[i]$ est le nombre total de souvenirs achetés du type i .